

im2col 설명

딥러닝, 특히 합성곱 신경망(CNN)에서 자주 등장하는 **im2col**은 이름 그대로 **Image to Column**, 즉 "이미지를 행렬(열)로 펼치는 기술"을 의미합니다.

CNN에서 필터를 슬라이딩하며 계산하는 과정(합성곱)을 컴퓨터가 가장 잘하는 '**행렬 곱셈**' 형태로 바꾸어 레이저처럼 빠르게 처리하기 위해 사용합니다.

이해하기 쉽게 작동 원리와 이유를 단계별로 나누어 설명해 드릴게요!

1. 왜 굳이 펼칠까요? (배경)

컴퓨터는 이미지 위에서 필터를 한 칸씩 움직이며(슬라이딩) 곱하고 더하는 복잡한 반복문(for-loop)을 싫어합니다. 속도가 아주 느려지기 때문이죠.

반면, 컴퓨터(특히 GPU)는 거대한 두 행렬을 한 번에 곱하는 연산(GEMM)을 기가 막히게 빠르게 처리합니다.

im2col의 핵심 아이디어:

"복잡하게 필터를 굴리지 말고, 이미지와 필터를 미리 행렬 형태로 다 쪼개고 펼쳐서
행렬 곱셈 한 방으로 끝내자!"

2. im2col은 어떻게 작동하나요? (예시)

여기에 **3x3 크기의 이미지**가 있고, **2x2 크기의 필터**로 합성곱을 한다고 가정해 봅시다. 필터는 총 4번 움직이게 됩니다.

① 이미지에서 필터가 찍는 '영역'을 찾습니다.

필터가 이미지를 훑을 때 거치는 4가지 영역(패치)이 있습니다.

1. **첫 번째 영역 (좌측 상단):** 이미지의 [1행 1열, 1행 2열, 2행 1열, 2행 2열]
2. **두 번째 영역 (우측 상단):** 이미지의 [1행 2열, 1행 3열, 2행 2열, 2행 3열]
3. **세 번째 영역 (좌측 하단):** 이미지의 [2행 1열, 2행 2열, 3행 1열, 3행 2열]
4. **네 번째 영역 (우측 하단):** 이미지의 [2행 2열, 2행 3열, 3행 2열, 3행 3열]

② 각 영역을 길게 한 줄(세로 열)로 펼칩니다.

- 첫 번째 영역 데이터 4개를 세로로 길게 [] 채웁니다.
- 두 번째 영역 데이터 4개도 그 옆 세로 줄에 채웁니다.

- 이렇게 4개의 영역을 모두 펼쳐서 옆으로 붙이면, 원래 3x3이었던 이미지가 **거대한 2D 행렬**로 변신합니다.

③ 필터도 한 줄로 펼쳐서 곱합니다.

2x2 크기의 필터도 가로로 길게 한 줄로 쪽 펼칩니다.

이제 [펼쳐진 필터 행렬] × [im2col로 펼쳐진 이미지 행렬]을 수행하면, 단 한 번의 행렬 곱셈으로 합성곱 연산이 끝납니다! 결과로 나온 행렬을 다시 원래 원했던 출력 크기(여기서는 2x2)로 모양만 바꿔주면(Reshape) 최종 출력이 됩니다.

3. 장점과 단점

- **장점 (압도적인 속도):** 컴퓨터가 제일 잘하는 표준 행렬 곱셈 알고리즘을 그대로 쓸 수 있어서 연산 속도가 엄청나게 빨라집니다. modern 딥러닝 프레임워크(PyTorch, TensorFlow 등)의 가속 비결 중 하나입니다.
- **단점 (메모리 낭비):** 위의 예시를 보면 이미지의 특정 픽셀(예: 2행 2열)이 첫 번째, 두 번째, 세 번째, 네 번째 영역에 모두 중복해서 들어갑니다. 즉, 겹치는 영역의 데이터를 중복해서 저장하기 때문에 **메모리를 많이 차지합니다.**

요약하자면

- im2col은 "for문 돌면서 필터 찍기 귀찮으니, 필터가 찍을 자리를 미리 다 행렬로 펼쳐놓고 한 방에 곱하자!"는 기술입니다.
- 메모리를 조금 희생하는 대신, **속도를 극한으로 끌어올리는** 현대 딥러닝의 필수 아이디어라고 보시면 됩니다.

① 이미지에서 필터가 찍는 '영역'을 찾습니다.

필터가 이미지를 훑을 때 거치는 4가지 영역(패치)이 있습니다.

첫 번째 영역 (좌측 상단): 이미지의 [1행 1열, 1행 2열, 2행 1열, 2행 2열]

텍스트로만 보면 조금 추상적이죠? 이번에는 구체적인 숫자(1부터 9까지)와 **가상의 필터**를 사용해서, 행렬이 실제로 어떻게 변하고 곱해지는지 눈으로 보듯 아주 자세하게 쪼개어 설명해 드릴게요.

0. 오늘의 주인공들 (세팅)

- **입력 이미지 (3x3 크기):**

1

2

3

4

5

6

7

8

9

...

- **필터 (2x2 크기):** 편의상 문자로 둘게요.

a

b

c

d

...

원래 합성곱 연산이라면 이 필터를 이미지 좌측 상단에 대고 $(1*a)+(2*b)+(4*c)+(5*d)$ 를 계산한 뒤, 옆으로 한 칸 이동해서 또 계산해야 합니다. 컴퓨터는 이 '이동하며 계산하기'를 아주 싫어 하죠.

이제 `im2col`이 이 작업을 어떻게 행렬 연산으로 바꾸는지 단계별로 보겠습니다.

1단계: 이미지에서 필터가 찍는 4가지 영역 추출하기

필터(2x2)가 3x3 이미지 위를 움직이면 총 4개의 영역을 보게 됩니다. 각 영역에 어떤 숫자들이 들어있는지 뽑아봅시다.

1. **영역 ① (좌측 상단):** 1, 2, 4, 5
2. **영역 ② (우측 상단):** 2, 3, 5, 6
3. **영역 ③ (좌측 하단):** 4, 5, 7, 8
4. **영역 ④ (우측 하단):** 5, 6, 8, 9

2단계: 각 영역을 '세로 한 줄(Column)'로 펼쳐서 합치기

이게 `im2col`(Image to Column)의 핵심입니다. 위에서 뽑은 4개의 영역을 각각 **세로로 길게 펼친 뒤, 옆으로 차례대로 이어 붙여서** 새로운 거대한 행렬을 만듭니다.

- **영역 ①**을 세로로: [1, 2, 4, 5]
- **영역 ②**를 세로로: [2, 3, 5, 6]

- 영역 ③을 세로로: [4, 5, 7, 8]
- 영역 ④을 세로로: [5, 6, 8, 9]

이걸 옆으로 붙이면 아래와 같은 **변신한 이미지 행렬**이 완성됩니다.

영역 ①	영역 ②	영역 ③	영역 ④
1	2	4	5
2	3	5	6
4	5	7	8
5	6	8	9

...

💡 **주목!** > 숫자 2, 4, 5, 6, 8 같은 애들이 여러 영역에 걸쳐서 중복으로 들어가 있는 게 보이시죠? 이 때문에 **메모리를 더 많이 쓰게 된다고** 한 것입니다.

3단계: 필터도 가로로 펼쳐서 곱하기

이제 필터(2x2)도 한 줄로 쪽 펼칩니다. 가로로 길게 [a, b, c, d] 로 만듭니다.

마지막으로 컴퓨터가 가장 잘하는 [행렬 곱셈]을 수행합니다.

Plaintext



[필터 행렬]	X	[변신한 이미지 행렬]																
[a, b, c, d]	X	<table> <tr><td> 1 </td><td>2 </td><td>4 </td><td>5 </td></tr> <tr><td> 2 </td><td>3 </td><td>5 </td><td>6 </td></tr> <tr><td> 4 </td><td>5 </td><td>7 </td><td>8 </td></tr> <tr><td> 5 </td><td>6 </td><td>8 </td><td>9 </td></tr> </table>	1	2	4	5	2	3	5	6	4	5	7	8	5	6	8	9
1	2	4	5															
2	3	5	6															
4	5	7	8															
5	6	8	9															

행렬 곱셈 공식(가로 줄 × 세로 줄)에 의해 결과는 다음과 같이 딱 한 줄로 나옵니다.

- **결과 행렬 (1x4 크기):**

[(1*a + 2*b + 4*c + 5*d), (2*a + 3*b + 5*c + 6*d), (4*a + 5*b + 7*c + 8*d), (5*a + 6*b + 8*c + 9*d)]

잘 보시면, 원래 우리가 원했던 합성곱 연산 결과 4개가 순서대로 다 계산되어 들어가 있습니다!

4단계: 원래 모양으로 바꾸기 (Reshape)

컴퓨터는 이 한 줄짜리 결과(1x4)를 우리가 최종적으로 원하는 2D 이미지 형태인 **2x2 크기**로 모양만 톡 바꾸어 줍니다.

결과 ①

결과 ②

결과 ③

결과 ④

...

이렇게 하면 복잡한 반복문 없이, 깔끔하게 합성곱 연산이 마무리됩니다.